Neulab
CAREER ACCELERATOR

Neulab Career Accelerator

Javascript language of the web

Practical, real-world training for modern full-stack developers.

Content overview

01

JS - what & why;
Brief history

02

Code structure & syntax

03

Variables & data types

04

Type Conversions

05

Operators, control-flow statements

06

Functions

07

Debugging & code conventions

JavaScript - what & why; Brief history

- Interpreted, single-threaded, garbage collected, multi-paradigm, dynamic
- Initially developed at Sun Microsystems, 1995 by Brandon Eich
- Core web technology alongside HTML and CSS
- ECMAScript and evolution
- Runs in ***every*** browser; has dedicated server-side runtimes (Node.js, Deno, Bun)

MDN *(Mozilla developer network)*

- The go-to place for official ECMAScript-, CSS-, HTML- and web-related references

  https://developer.mozilla.org/en-US/

Code structure & syntax

- C-like syntax and keywords (though actual semantics may differ greatly)
- Line-by-line execution
- Optional semicolon (;)
- Line and block Comments

Variables & data types

- Old (Pre-ES5) declaration used **var**
- **var** declares *functionally-scoped* variables
- Modern JS (ES5+) allows *block-scoped* variable declarations with either:
  - **let** - mutable variables
  - **const** - immutable variables

Variables & data types

Primitives	Referential
string (String)	object (Object)
boolean (Boolean)	
number (Number)	
null	
undefined	
bigint (BigInt)	
symbol (Symbol)	

Variables & data types

	typeof return value
object	“object”
string (String)	“string”
boolean (Boolean)	“boolean”
number (Number)	“number”
null	“object”
undefined	“undefined”
bigint (BigInt)	“bigint”
symbol (Symbol)	“symbol”

Type conversions

- Typecasts can be *explicit* and *implicit*
  - explicit casts are achieved with primitive type object wrappers (Number, String, etc)

Type conversions

- In practice, it is useful to:
  - prepend **+** to convert a (valid numeric) string to number instead of Number(<string value>)
  - Append an “” to convert a value to string (though the toString method can (preferably) be used too)
  - prepend **!!** to get the boolean value of an expression (first **!** negates the value, implicitly converting to *boolean*, second **!** negates again to get the truthy value)

Type conversions

Falsy values (evaluate to *false* when converted to *boolean*)	
null	false
undefined	“”
NaN	0n
0	-0
document.all	

Operators, control-flow statements

- JS uses already familiar C-style statements
  - *if-else* (conditional branching)
  - *for* (incremental loop)
  - *while/do-while* (conditional loop)
  - *switch* (multi-case conditional branching)

Operators, control-flow statements

- JS supports the already familiar C-style operators
  - Logical (**&&** and **||**)
  - Arithmetic
  - Bitwise
  - Comparison
    - **==** compares implicitly converted values
    - **===** compares ***both*** value and type
    - Object-to-object comparison evaluates object ***references, not actual values***

Functions

- Hoisting
- Default values
- Arrow functions
- Closure
- Pure functions and side effects

Debugging and code quality

- Code conventions
- Comments
- *eslint* + *prettier* as de-facto linter & formatter
- Debugging
- Prefer **readable** over **concise and inlined** .... PLEASE

Do you have questions?

Neulab
CAREER ACCELERATOR

Thank you!

Neulab
CAREER ACCELERATOR